

Problem 1. (*Finite difference with convolution*) $X \in \mathbb{R}^{m \times n}$, $Y \in \mathbb{R}^{2 \times m \times n}$ 일 때,

$$Y_{1,i,j} = X_{i+1,j} - X_{i,j}$$

$$Y_{2,i,j} = X_{i,j+1} - X_{i,j}$$

이 경우, output channel이 여러 개인 상황이고, filter $w \in \mathbb{R}^{2 \times 3 \times 3}$ 는 정확히는 $w \in \mathbb{R}^{2 \times 1 \times 3 \times 3}$ (output, input, vertical, horizontal)이어야 한다. 위의 조건을 만족하는 w 는 다음과 같다.

$$w_{1,2,2} = -1, w_{1,3,2} = 1, \text{otherwise } w_{1,i,j} = 0$$

$$w_{2,2,2} = -1, w_{2,2,3} = 1, \text{otherwise } w_{2,i,j} = 0$$

zero padding이 1이므로, 원래의 image와 x,y 방향 길이가 같다.

Problem 2. (*Average pooling as convolution*) $X \in \mathbb{R}^{C \times m \times n}$, $Y \in \mathbb{R}^{C \times (m/k) \times (n/k)}$ 일 때, AvgPool2d 연산

$$Y_{c,i,j} = \frac{1}{k^2} \sum_{a=1}^k \sum_{b=1}^k X_{c,k(i-1)+a,k(j-1)+b} \quad (i = 1, 2, \dots, (m/k), j = 1, 2, \dots, (n/k))$$

일 때, 이는 kernel size가 k이고 stride도 k인 것이다. (그래야 출력되는 텐서의 차원이 맞는다.)

위의 조건을 만족하는 filter $w \in \mathbb{R}^{C \times C \times k \times k}$ 는

$$w_{c,d,i,j} = \frac{1}{k^2} \quad \text{if } d = c$$

$$w_{c,d,i,j} = 0 \quad \text{if } d \neq c$$

가 된다.

Problem 3. (*RGB to greyscale mapping with 1×1 convolution*) $X \in \mathbb{R}^{3 \times m \times n}$, $Y \in \mathbb{R}^{1 \times m \times n}$ 일 때, RGB 픽셀을 회색조(greyscale)로 변환하는 연산(Y value)은

$$Y_{i,j} = 0.299X_{1,i,j} + 0.587X_{2,i,j} + 0.114X_{3,i,j} \quad (\text{순서대로 R,G,B})$$

이다. 이 경우, output channel은 한 개인 반면 input channel이 여러 개인 상황이다. 그리고 위의 조건을 만족하는 filter $w \in \mathbb{R}^{3 \times 1 \times 1}$ (정확히는, 문제 1과 같은 이유에서 $w \in \mathbb{R}^{1 \times 3 \times 1 \times 1}$ 이다.)는

$$w_{1,1,1} = 0.299, w_{2,1,1} = 0.587, w_{3,1,1} = 0.114$$

가 된다.

정답은 거의 자명하지만, 이러한 1×1 convolution을 사용하는 이유가 존재한다. 이렇게 여러 필터에서 가중치를 부여하여 하나의 값(single channel)으로 반환하는 과정에서 차원의 축소가 일어난다. 이러한 차원 축소로 인하여 연산량에서 큰 이득을 볼 수 있고, 따라서 보다 더 깊은 신경망을 사용할 수 있다. 학습을 지속할 수 있는 적당한 차원으로서의 축소가 필요한데, 이러한 구조는 **bottleneck**이라고 한다.

Problem 4. $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ 이 단조증가하는 활성화함수라고 하고, $\rho : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{k \times l}$ 은 maxpool 연산이라고 하자. 이 때 모든 $X \in \mathbb{R}^{m \times n}$ 에 대해

$$\sigma(\rho(X)) = \rho(\sigma(X))$$

가 성립함을 보여라.

Proof.

X 를 $m \times n$ 행렬로 생각하지 말고 $m \times n$ 개의 원소가 있는 집합이라고 생각하자. 해당 집합의 임의의 공집합이 아닌 순서 있는 부분집합(ordered basis와 같이 집합임에도 불구하고 이를 순서가 있는 집합으로 생각하자.) $E \in X$ 에 대해서, σ 가 단조증가함수이므로 순서집합 E 와 $\sigma(E)$ 의 크기 순서는 동일하다. 따라서 maxpool 연산이 구체적으로 어떠한 filter로 pooling이 되든지, 먼저 활성화함수를 취하고 그 중 최댓값을 고르는 것과 최댓값을 먼저 고르고 그 값에 활성화함수를 취하는 것은 동등하다. $\square$

Problem 5. (*Non-CE loss function*) MNIST 데이터 집합에 대해서, 모델은 LR(로지스틱 회귀)에서 사용했던 대로

$$f_{a,b}(x) = \begin{bmatrix} 1/(1 + e^{(a^\top x + b)}) \\ 1/(1 + e^{-(a^\top x + b)}) \end{bmatrix}$$

로 생각하되, 최소화할 목적함수를 KL-divergence 대신 sum-of-squares loss로 사용하자. 즉,

$$\underset{a \in \mathbb{R}^p, b \in \mathbb{R}}{\text{minimise}} \sum_{i=1}^N D_{\text{KL}}(\mathcal{P}(Y_i) \parallel f_{a,b}(X_i))$$

대신

$$\underset{a \in \mathbb{R}^p, b \in \mathbb{R}}{\text{minimise}} \frac{1}{N} \sum_{i=1}^N \|\mathcal{P}(Y_i) \parallel f_{a,b}(X_i)\|^2$$

를 수행한다. 물론,

$$\mathcal{P}(Y_i) = \begin{cases} [1 & 0]^\top & \text{if } y = -1 \\ [0 & 1]^\top & \text{if } y = 1 \end{cases}$$

이다. 이 때 서로 다른 두 목적함수에 대해 성능을 비교해보자.

KL-divergence의 정의에 의해, 첫 번째 경우는 손실함수 $l(z) = -\log(1/(1 + e^{-z}))$ (log-sigmoid)에 대해

$$\underset{a \in \mathbb{R}^p, b \in \mathbb{R}}{\text{minimise}} \frac{1}{N} \sum_{i=1}^N l(Y_i(a^\top X_i + b))$$

를 수행하는 것이고, 두 번째 경우는 손실함수 $l(z, y) = \frac{1}{2}(1 - y)[(1 - \frac{1}{1+e^z})^2 + (\frac{1}{1+e^{-z}})^2] + \frac{1}{2}(1 + y)[(1 - \frac{1}{1+e^{-z}})^2 + (\frac{1}{1+e^z})^2]$ 에 대해

$$\underset{a \in \mathbb{R}^p, b \in \mathbb{R}}{\text{minimise}} \frac{1}{N} \sum_{i=1}^N l(a^\top X_i + b, Y_i)$$

를 수행하는 것이다.

손실함수는 원래의 최소화 함수와 상당히 다르게 생긴 것처럼 보이나, 실제로는 $Y_i = -1, 1$ 을 대입하면 원래 목적함수의 형태가 나타난다. 이제 각각을 학습시키면, KL-divergence의 경우 92.57%의 정확도를 보이지만 sum-of-squares loss의 경우 50.13%의 정확도를 보이는 것을 확인할 수 있다. 위의 결과는 파일의 맨 뒤에 첨부한다.

Problem 6. (*Backprop for MLP*) $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ 가 미분가능한 활성화함수라고 할 때 다음과 같은 다중 퍼셉트론이 있다고 하자.

$$\begin{aligned} y_L &= A_L y_{L-1} + b_L \\ y_{L-1} &= \sigma(A_{L-1} y_{L-2} + b_{L-1}) \\ &\dots \\ y_2 &= \sigma(A_2 y_1 + b_2) \\ y_1 &= \sigma(A_1 x + b_1) \end{aligned}$$

여기서 $x \in \mathbb{R}^{n_0}$, $A_l \in \mathbb{R}^{n_l \times n_{l-1}}$, $b_l \in \mathbb{R}^{n_l}$, $n_L = 1$ 이다. $y_0 := x$ 라고 할 때, 다음을 보여라.

(a)

$$\begin{aligned} \frac{\partial y_L}{\partial b_L} &= I, \quad \frac{\partial y_L}{\partial y_{L-1}} = A_L \\ \frac{\partial y_l}{\partial b_l} &= \text{diag}(\sigma'(A_l y_{l-1} + b_l)), \quad \text{for } l = 1, 2, \dots, L-1 \\ \frac{\partial y_l}{\partial y_{l-1}} &= \text{diag}(\sigma'(A_l y_{l-1} + b_l)) A_l, \quad \text{for } l = 2, \dots, L-1 \end{aligned}$$

Proof.

$$\begin{aligned} y_{L,1} = \sum_j (A_L)_{1,j} y_{L-1,j} + b_{L,1} \text{에서 } \frac{\partial y_{L,1}}{\partial b_{L,1}} &= 1, \quad \frac{\partial y_{L,i}}{\partial y_{L-1,j}} = (A_L)_{1,j} \\ y_{l,i} = \sigma(\sum_j (A_l)_{i,j} y_{l-1,j} + b_{l,i}) \text{에서 } \frac{\partial y_{l,i}}{\partial b_{l,i}} &= \sigma' \delta_{i,i}, \quad \frac{\partial y_{l,i}}{\partial y_{l-1,j}} = \sigma' (A_l)_{i,j} \end{aligned}$$

이를 다시 항별로 행렬에 대입하면 위의 결과를 얻는다. (참고 : $[\delta_{i,j}] = I$ 이고, diag 가 행렬의 앞에 곱해지면 행마다 대각행렬의 원소가 각각(element-wise) 곱해진다.) $\square$

(b) $\frac{\partial y_L}{\partial A_l} \in \mathbb{R}^{n_l \times n_{l-1}}$ 을 다음과 같이 정의하자.

$$\left(\frac{\partial y_L}{\partial A_l} \right)_{ij} = \frac{\partial y_L}{\partial (A_l)_{ij}}$$

이 때 다음을 증명하라.

$$\begin{aligned} \frac{\partial y_L}{\partial A_L} &= y_{L-1}^\top \\ \frac{\partial y_L}{\partial A_l} &= \text{diag}(\sigma'(A_l y_{l-1} + b_l)) \left(\frac{\partial y_L}{\partial y_l} \right)^\top y_{l-1}^\top, \quad l = 1, 2, \dots, L-1 \end{aligned}$$

Proof.

먼저 $y_{L,1} = \sum_j A_{L,1,j} y_{L-1,j} + b_{L,1}$ 이므로

$$\begin{aligned}\frac{\partial y_L}{\partial (A_L)_{1,j}} &= y_{L-1,j} \\ \frac{\partial y_L}{\partial A_L} &= \begin{pmatrix} y_{L-1,1} & \dots & y_{L-1,n_{L-1}} \end{pmatrix} = y_{L-1}^\top\end{aligned}$$

다음으로, $y_{l,i} = \sigma(\sum_j (A_l)_{i,j} y_{l-1,j} + b_{l,i})$ 에서

$$\begin{aligned}\frac{\partial y_L}{\partial (A_l)_{i,j}} &= \frac{\partial y_L}{\partial y_l} \frac{\partial y_l}{\partial (A_l)_{i,j}} \\ &= \frac{\partial y_L}{\partial y_l} \begin{pmatrix} 0 \\ \vdots \\ (y_{l-1})_j \sigma' \\ \vdots \\ 0 \end{pmatrix} \\ &= \sigma' \cdot (y_{l-1})_j \frac{\partial y_L}{\partial y_{l,i}} \\ \frac{\partial y_L}{\partial A_l} &= \sigma' \begin{pmatrix} \frac{\partial y_L}{\partial y_{l,1}} \\ \vdots \\ \frac{\partial y_L}{\partial y_{l,n_l}} \end{pmatrix} \begin{pmatrix} (y_{l-1})_1 & \dots & (y_{l-1})_{n_{l-1}} \end{pmatrix} \\ &= \text{diag}(\sigma'(A_l y_{l-1} + b_l)) \left(\frac{\partial y_L}{\partial y_l} \right)^\top y_{l-1}^\top\end{aligned}$$

이므로 위의 결과를 얻는다. □

Problem 7. (*LeNet5 and C3-layer architecture*) 현재 사용하고 있는 LeNet5는 모든 레이어에서 모든 채널에 convolution 연산이 실행되지만, 최초의 LeNet5 구조에서는 C3-layer를 통과하는 과정에서 6개의 input 채널 중 3개씩 6번, 4개씩 9번, 그리고 마지막으로 전 채널에 필터를 1번 convolution하여 16개의 output 채널을 얻는 과정을 거쳤다. 현재 사용되는 LeNet5의 parameter 개수와 최초에 사용한 LeNet5의 parameter 개수 차이를 구해보자. 단, C3 layer을 제외하면 그대로라고 가정한다. Google colab을 통해 구한 parameter의 개수는 original과 modern 각각 60806, 61706개이므로, original의 parameter 개수가 modern에 비해 900개 적다는 결론이 나온다. 이 결과가 실제와 합치하는가?

modern LeNet5의 경우, 다른 C 레이어와 마찬가지로 단순히 존재하는 모든 input channel과 filter가 convolution 하는 과정을 거치는 것이므로 이 과정에서 생성되는 parameter는 $16 \times (6 \times 5^2 + 1) = 2416$ 개이다.

한편 original LeNet5의 경우, 6개의 input channel 중 16개의 조합으로 각각 하나씩 새로운 channel을 만들어 합치는 과정을 거치는 것이므로 이 과정에서 생성되는 parameter는 $6 \times (3 \times 5^2 + 1) + 9 \times (4 \times 5^2 + 1) + 1 \times (6 \times 5^2 + 1) = 1516$ 개이다.

따라서, 이론적으로 계산된 실제 감소된 parameter 수와 colab을 통해 계산된 parameter 개수의 차이는 일치한다. 5번과 마찬가지로 위의 구체적인 결과는 파일의 맨 뒤에 첨부한다.

참고사항 1 과제 코드에서 구현된 LeNet5는 "Gradient-based learning applied to document recognition" (Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, 1998) 에서 구현된 실제 original LeNet5와는 parameter 개수에서 약간 차이가 있다. (각각 60806개, 60000개) 이러한 현상은 왜 발생하는가?

그 이유는 과제 코드에서 구현된 LeNet5 model이 완전하지 못하기 때문이다. 먼저, 실제 original LeNet5는 S2, S4 layer에서 subsampling을 통해 average pooling을 하는 과정에서도 trainable parameter가 생성되는데, 이는 average pooling이 우리가 알고 있던 의미와 다른 것이 아니라 S2, S4 layer을 거친 직후 sigmoid activation을 다시 수행하는 과정에서 weight, bias parameter가 각각 한 개씩 추가되기 때문이다.

또한, 과제 스펙에서는 F7(output) layer이 nn.linear로 구현되었기 때문에 앞에서의 convolutional layer과 마찬가지로 parameter가 생성되는 것으로 간주되는데, 실제 LeNet5에서는 F6에서 output layer로 이동하는 과정에서 Gaussian connection이라는 방법으로 값이 정해진다. 즉 parameter가 아니다.

이러한 이유에서 실제 논문에서와 parameter 개수의 차이가 발생했던 것으로, 이 값을 보정하면 정확히 60,000개의 parameter가 있음을 확인할 수 있다. 구체적으로 어떻게 생겼는지에 대해서 계산해보기로 한다.

(정확한 original LeNet5 parameter 개수)

C1 layer : $32 \times 32, f = 5, k = 6$ 에서 $6 \times (5^2 + 1) = \mathbf{156}$ 개, $6 \times 28 \times 28$ 으로 변경

S2 layer : filter:2, stride:2를 이용해 subsampling, average pooling을 하는 과정에서 6개의 채널 각각에 1개의 trainable weight과 bias를 추가하므로 $6 \times 2 = \mathbf{12}$ 개, $6 \times 14 \times 14$ 로 변경

C3 layer : 앞에서 계산한 결과에 의해 **1516**개, $16 \times 10 \times 10$ 으로 변경

S4 layer : filter:2, stride:2를 이용해 subsampling, average pooling을 하는 과정에서 16개의 채널 각각에 1개의 trainable weight과 bias를 추가하므로 $16 \times 2 = \mathbf{32}$ 개, $16 \times 5 \times 5$ 로 변경

C5 layer : full connection을 진행, $120 \times (16 \times 5 \times 5 + 1) = \mathbf{48120}$ 개, 현재의 image를 통째로 120개의 같은 size의 filter와 convolution하여 $120 \times 1 \times 1$ 의 새로운 image로 변경

F6 layer : full connection을 진행, $84 \times (120 + 1) = \mathbf{10164}$ 개, 현재의 image를 통째로 84개의 같은 size의 filter와 convolution하여 $84 \times 1 \times 1$ 의 새로운 image로 변경

output layer : Gaussian connection을 진행, 생성되는 parameter **없음**.

합계 : $156 + 12 + 1516 + 32 + 48120 + 10164 = \mathbf{60000}$ 개의 parameter가 생성.

참고사항 2 왜 C3 layer에서 모든 channel을 filter와 convolution하지 않았는가? 이는 LeCun이 처음 논문을 냈을 당시 이러한 구조를 설계했던 의도가 network에 내재될 수 있는 대칭성(symmetry)을 파괴하기 위함이었기 때문이다. 그러나 현재는 가중치와 bias의 랜덤 초기화(random initialisation)가 이러한 대칭성을 파괴하기에 충분하다고 여긴다.

MATHDNN_HW4

October 5, 2021

Problem 5

```
[1]: import torch
import torch.nn as nn
from torch.optim import Optimizer
from torch.utils.data import DataLoader

# torchvision: popular datasets, model architectures, and common image
# → transformations for computer vision.
from torchvision import datasets
from torchvision.transforms import transforms

from random import randint
from random import shuffle
import numpy as np
import matplotlib.pyplot as plt

'''
Step 1: Prepare dataset
'''

# Use data with only 4 and 9 as labels: which is hardest to classify
label_1, label_2 = 4, 9

# MNIST training data
train_set = datasets.MNIST(root='./mnist_data/', train=True,
    → transform=transforms.ToTensor(), download=True)

# Use data with two labels
idx = (train_set.targets == label_1) + (train_set.targets == label_2)
train_set.data = train_set.data[idx]
train_set.targets = train_set.targets[idx]
train_set.targets[train_set.targets == label_1] = -1
train_set.targets[train_set.targets == label_2] = 1

# MNIST testing data
test_set = datasets.MNIST(root='./mnist_data/', train=False,
    → transform=transforms.ToTensor())
```

```
# Use data with two labels
idx = (test_set.targets == label_1) + (test_set.targets == label_2)
test_set.data = test_set.data[idx]
test_set.targets = test_set.targets[idx]
test_set.targets[test_set.targets == label_1] = -1
test_set.targets[test_set.targets == label_2] = 1
```

Downloading <http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz>
 Downloading <http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz> to
 ./mnist_data/MNIST/raw/train-images-idx3-ubyte.gz

```
0%|          | 0/9912422 [00:00<?, ?it/s]
```

Extracting ./mnist_data/MNIST/raw/train-images-idx3-ubyte.gz to
 ./mnist_data/MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz>
 Downloading <http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz> to
 ./mnist_data/MNIST/raw/train-labels-idx1-ubyte.gz

```
0%|          | 0/28881 [00:00<?, ?it/s]
```

Extracting ./mnist_data/MNIST/raw/train-labels-idx1-ubyte.gz to
 ./mnist_data/MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz>
 Downloading <http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz> to
 ./mnist_data/MNIST/raw/t10k-images-idx3-ubyte.gz

```
0%|          | 0/1648877 [00:00<?, ?it/s]
```

Extracting ./mnist_data/MNIST/raw/t10k-images-idx3-ubyte.gz to
 ./mnist_data/MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz>
 Downloading <http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz> to
 ./mnist_data/MNIST/raw/t10k-labels-idx1-ubyte.gz

```
0%|          | 0/4542 [00:00<?, ?it/s]
```

Extracting ./mnist_data/MNIST/raw/t10k-labels-idx1-ubyte.gz to
 ./mnist_data/MNIST/raw

/usr/local/lib/python3.7/dist-packages/torchvision/datasets/mnist.py:498:
 UserWarning: The given NumPy array is not writeable, and PyTorch does not
 support non-writeable tensors. This means you can write to the underlying
 (supposedly non-writeable) NumPy array using the tensor. You may want to copy
 the array to protect its data or make it writeable before converting it to a
 tensor. This type of warning will be suppressed for the rest of this program.

```
(Triggered internally at /pytorch/torch/csrc/utils/tensor_numpy.cpp:180.)
return torch.from_numpy(parsed.astype(m[2], copy=False)).view(*s)
```

```
[6]: '''
Step 2: Define the neural network class
'''
class LR(nn.Module) :
    '''
    Initialize model
    input_dim : dimension of given input data
    '''
    # MNIST data is 28x28 images
    def __init__(self, input_dim=28*28) :
        super().__init__()
        self.linear = nn.Linear(input_dim, 1, bias=True)

    ''' forward given input x '''
    def forward(self, x) :
        return self.linear(x.float().view(-1, 28*28))
```

```
[16]: '''
Step 3: Create the model, specify loss function and optimizer.
'''
model = LR() # Define a Neural Network Model
model_new = LR()

def logistic_loss(output, target):
    return -torch.nn.functional.logsigmoid(target*output)

def logistic_loss_new(output, target): # target : -1 or 1, output : some real
    ↪ number, componentwisely returned
    return 0.5*(1-target)*((1-torch.nn.functional.logsigmoid(-output))**2+torch.
    ↪ nn.functional.logsigmoid(output)**2)+0.5*(1-target)*((1-torch.nn.functional.
    ↪ logsigmoid(output))**2+torch.nn.functional.logsigmoid(-output)**2)

loss_function = logistic_loss
loss_function_new = logistic_loss_new
    ↪ # Specify loss function
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4) # specify SGD with
    ↪ learning rate
optimizer_new = torch.optim.SGD(model.parameters(), lr=1e-4)
```

Using KL-divergence

```
[17]: '''
Step 4: Train model with SGD
'''
import time
```

```

start = time.time()
for _ in range(1000) :
    # Sample a random data for training
    ind = randint(0, len(train_set.data)-1)
    image, label = train_set.data[ind], train_set.targets[ind]

    # Clear previously computed gradient
    optimizer.zero_grad()

    # then compute gradient with forward and backward passes
    train_loss = loss_function(model(image), label.float())
    train_loss.backward()

    #(This syntax will make more sense once we learn about minibatches)

    # perform SGD step (parameter update)
    optimizer.step()

end = time.time()
print(f"Time elapsed in training is: {end-start}")

```

Time elapsed in training is: 0.25348496437072754

```

[18]: '''
Step 5: Test model (Evaluate the accuracy)
'''
test_loss, correct = 0, 0
misclassified_ind = []
correct_ind = []

# Evaluate accuracy using test data
for ind in range(len(test_set.data)) :

    image, label = test_set.data[ind], test_set.targets[ind]

    # evaluate model
    output = model(image)

    # Calculate cumulative loss
    test_loss += loss_function(output, label.float()).item()

    # Make a prediction
    if output.item() * label.item() >= 0 :
        correct += 1
        correct_ind += [ind]
    else:
        misclassified_ind += [ind]

```

```
# Print out the results
print('[Test set] Average loss: {:.4f}, Accuracy: {}/{} ({:.2f}%)\\n'.format(
    test_loss / len(test_set.data), correct, len(test_set.data),
    100. * correct / len(test_set.data)))
```

[Test set] Average loss: 10.2567, Accuracy: 1843/1991 (92.57%)

Using sum-of-squares loss

```
[19]: import time
start = time.time()
for _ in range(1000) :
    # Sample a random data for training
    ind = randint(0, len(train_set.data)-1)
    image, label = train_set.data[ind], train_set.targets[ind]

    # Clear previously computed gradient
    optimizer_new.zero_grad()

    # then compute gradient with forward and backward passes
    train_loss = loss_function_new(model_new(image), label.float())
    train_loss.backward()

     #(This syntax will make more sense once we learn about minibatches)

    # perform SGD step (parameter update)
    optimizer_new.step()

end = time.time()
print(f"Time elapsed in training is: {end-start}")
```

Time elapsed in training is: 0.5263056755065918

```
[20]: '''
Step 5: Test model (Evaluate the accuracy)
'''
test_loss, correct = 0, 0
misclassified_ind = []
correct_ind = []

# Evaluate accuracy using test data
for ind in range(len(test_set.data)) :

    image, label = test_set.data[ind], test_set.targets[ind]

    # evaluate model
    output = model_new(image)
```

```

# Calculate cumulative loss
test_loss += loss_function_new(output, label.float()).item()

# Make a prediction
if output.item() * label.item() >= 0 :
    correct += 1
    correct_ind += [ind]
else:
    misclassified_ind += [ind]

# Print out the results
print('[Test set] Average loss: {:.4f}, Accuracy: {}/{} ({:.2f}%)\n'.format(
    test_loss / len(test_set.data), correct, len(test_set.data),
    100. * correct / len(test_set.data)))

```

[Test set] Average loss: 2856.8724, Accuracy: 998/1991 (50.13%)

Problem 7

```

[38]: import torch
import torch.nn as nn
from torch.optim import Optimizer
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import transforms

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

[39]: '''
Step 1:
'''

# MNIST dataset
train_dataset = datasets.MNIST(root='./mnist_data/',
                                train=True,
                                transform=transforms.ToTensor(),
                                download=True)

test_dataset = datasets.MNIST(root='./mnist_data/',
                               train=False,
                               transform=transforms.ToTensor())

```

```

[68]: '''
Step 2: LeNet5
'''

```

```

# Modern LeNet uses this layer for C3
class C3_layer_full(nn.Module):
    def __init__(self):
        super(C3_layer_full, self).__init__()
        self.conv_layer = nn.Conv2d(6, 16, kernel_size=5) # num. of input
        ↪ channel : 6, num. of output channel : 16, kernel(batch) size : 5*5

    def forward(self, x):
        return self.conv_layer(x)

# Original LeNet uses this layer for C3
class C3_layer(nn.Module):
    def __init__(self):
        super(C3_layer, self).__init__()
        self.ch_in_3 = [[0, 1, 2],
                        [1, 2, 3],
                        [2, 3, 4],
                        [3, 4, 5],
                        [0, 4, 5],
                        [0, 1, 5]] # filter with 3 subset of input channels
        self.ch_in_4 = [[0, 1, 2, 3],
                        [1, 2, 3, 4],
                        [2, 3, 4, 5],
                        [0, 3, 4, 5],
                        [0, 1, 4, 5],
                        [0, 1, 2, 5],
                        [0, 1, 3, 4],
                        [1, 2, 4, 5],
                        [0, 2, 3, 5]] # filter with 4 subset of input channels
        # put implementation here
        self.ch_in_6 = [[0, 1, 2, 3, 4, 5]]
        self.conv_layer_3 = nn.ModuleList([nn.Conv2d(3, 1, kernel_size=5) for _ in
        ↪ in range(6)]) # initialisation
        self.conv_layer_4 = nn.ModuleList([nn.Conv2d(4, 1, kernel_size=5) for _ in
        ↪ in range(9)])
        self.conv_layer_6 = nn.ModuleList([nn.Conv2d(6, 1, kernel_size=5)])

    def forward(self, x):
        # put implementation here
        self.ch_out_3 = torch.cat([self.conv_layer_3[i](x[:, self.ch_in_3[i], :
        ↪ , :]) for i in range(6)], 1) # dim = 1, you can determine the exact dimension
        ↪ to print self.ch.out.shape
        self.ch_out_4 = torch.cat([self.conv_layer_4[i](x[:, self.ch_in_4[i], :
        ↪ , :]) for i in range(9)], 1)
        self.ch_out_6 = torch.cat([self.conv_layer_6[i](x[:, self.ch_in_6[i], :
        ↪ , :]) for i in range(1)], 1)

```

```

        self.ch_out = torch.cat([self.ch_out_3, self.ch_out_4, self.ch_out_6],1)
        return self.ch_out

class LeNet(nn.Module) :
    def __init__(self) :
        super(LeNet, self).__init__()
        #padding=2 makes 28x28 image into 32x32
        self.C1_layer = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2),
            nn.Tanh()
        )
        self.P2_layer = nn.Sequential(
            nn.AvgPool2d(kernel_size=2, stride=2),
            nn.Tanh()
        )
        self.C3_layer = nn.Sequential(
            #C3_layer_full(), # If we use C3_layer_full instead of
            ↪C3_layer, we can get different total num. of trainable params, and different
            ↪accuracy. (98.50%)
            C3_layer(),
            nn.Tanh()
        )
        self.P4_layer = nn.Sequential(
            nn.AvgPool2d(kernel_size=2, stride=2),
            nn.Tanh()
        )
        self.C5_layer = nn.Sequential(
            nn.Linear(5*5*16, 120),
            nn.Tanh()
        )
        self.F6_layer = nn.Sequential(
            nn.Linear(120, 84),
            nn.Tanh()
        )
        self.F7_layer = nn.Linear(84, 10)
        self.tanh = nn.Tanh()

    def forward(self, x) :
        output = self.C1_layer(x)
        output = self.P2_layer(output)
        output = self.C3_layer(output)
        output = self.P4_layer(output)
        output = output.view(-1,5*5*16)
        output = self.C5_layer(output)
        output = self.F6_layer(output)
        output = self.F7_layer(output)
        return output

```

```
[69]: '''
      Step 3
      '''

model = LeNet().to(device)
loss_function = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-1)

# print total number of trainable parameters
param_ct = sum([p.numel() for p in model.parameters()])
print(f"Total number of trainable parameters: {param_ct}")
```

Total number of trainable parameters: 60806

In the original LeNet5, there are exactly 60,000 trainable parameters. (60806+12+32-850=60000)

It's because in the original LeNet5, average pooling actually contains two trainable parameters per one channel, and F7(output) layer does not contain such trainable parameters.

```
[70]: '''
      Step 4
      '''

train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
      ↪batch_size=100, shuffle=True)

import time
start = time.time()
for epoch in range(10) :
    print("{}th epoch starting.".format(epoch))
    for images, labels in train_loader :
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        train_loss = loss_function(model(images), labels)
        train_loss.backward()

        optimizer.step()
end = time.time()
print("Time ellapsed in training is: {}".format(end - start))
```

```
0th epoch starting.
1th epoch starting.
2th epoch starting.
3th epoch starting.
4th epoch starting.
5th epoch starting.
6th epoch starting.
7th epoch starting.
8th epoch starting.
9th epoch starting.
```

Time elapsed in training is: 586.6096761226654

```
[71]: '''  
Step 5  
'''  
test_loss, correct, total = 0, 0, 0  
  
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=100,   
→shuffle=False)  
  
for images, labels in test_loader :  
    images, labels = images.to(device), labels.to(device)  
  
    output = model(images)  
    test_loss += loss_function(output, labels).item()  
  
    pred = output.max(1, keepdim=True)[1]  
    correct += pred.eq(labels.view_as(pred)).sum().item()  
  
    total += labels.size(0)  
  
print('[Test set] Average loss: {:.4f}, Accuracy: {}/{} ({:.2f}%)\n'.format(  
    test_loss / total, correct, total,  
    100. * correct / total))
```

[Test set] Average loss: 0.0004, Accuracy: 9873/10000 (98.73%)